AI Firewall: pasarela de seguridad para aplicaciones con LLM

Trabajo de Fin de Grado – CFGS Desarrollo de Aplicaciones Multiplataforma (DAM)

Autores: Lucía Anguita Luna, Liam Bezanilla Alonso y Alexandru Ilie Gabriel

Documento: Actividad 1: Anteproyecto

1. Miembros del proyecto y dedicación inicial

El equipo lo forman tres personas. Todos participan en todas las áreas técnicas para que cada uno pueda defender el proyecto completo; Lucía Anguita Luna es la única Team Lead.

Miembro	Rol	Participación técnica
Lucía Anguita Luna	Team Lead: planificación, seguimiento, riesgos, relación con el tutor y memoria	RAG seguro, red teaming y métricas, frontend
Liam Bezanilla Alonso	Arquitecto técnico y responsable del núcleo de IA y seguridad	Gateway, capas de detección, revisión de código, apoyo en cloud
Alexandru Ilie Gabriel	Responsable de datos y de la base de datos no relacional, pruebas, documentación técnica y automatización del despliegue	DynamoDB y S3, ataques de prueba para el red teaming, documentos ficticios del RAG, Docker y docker-compose, CI/CD con GitHub Actions y Terraform guiado

Cada módulo tiene un responsable y un segundo que revisa el código y programa en parejas en las partes clave:

Módulo	Responsable	Segundo
Gateway y capas de detección	Liam	Lucía
RAG seguro y permisos por rol	Lucía	Liam
Red teaming, métricas e informe	Lucía	Alexandru
Infraestructura en AWS y despliegue	Liam	Alexandru
Base de datos, eventos y auditoría	Alexandru	Lucía

Módulo	Responsable	Segundo
Dashboard y chat de demostración	Lucía y Liam	Alexandru

Cada dos semanas se hará una sesión de enseñanza en la que cada miembro explica su parte al resto.

2. Idea del proyecto

2.1 Problema

Las empresas incorporan asistentes basados en modelos de lenguaje (LLM) para consultar documentación o atender a clientes. Esto introduce riesgos recogidos en el OWASP Top 10 for LLM Applications: prompt injection, fuga de datos personales o confidenciales, extracción de las instrucciones internas del modelo, agentes que ejecutan acciones no autorizadas y falta de trazabilidad. Muchas pymes no tienen recursos para defenderse de ellos ni para medir si su asistente es seguro.

2.2 Propuesta

AI Firewall es una pasarela de seguridad (gateway) en Python que se sitúa entre una aplicación y el modelo de lenguaje que usa. Analiza cada petición y cada respuesta, las permite, enmascara o bloquea según unas políticas, y registra el evento para su auditoría. Expone una API compatible con la de OpenAI, de modo que una aplicación existente se protege cambiando solo la dirección de destino (`base_url`).

Para demostrarlo y evaluarlo, el proyecto incluye un chatbot corporativo de ejemplo, un módulo de ataques automáticos y un panel web de administración.

2.3 Componentes

1. **Gateway con detección por capas**: reglas para ataques conocidos, detección y enmascarado de datos sensibles (DNI, IBAN, correo, claves de API), clasificador de IA para prompt injection y jailbreak, y filtro de salida que detecta fugas de datos o de las instrucciones internas.
2. **Asistente con RAG seguro**: chatbot de una empresa ficticia que responde con sus documentos internos y respeta los permisos por rol.
3. **Red teaming y métricas**: batería de ataques clasificados según OWASP que se lanza contra el asistente con y sin firewall, midiendo tasa de detección, falsos positivos y latencia añadida.
4. **Aplicación web**: dashboard de seguridad (eventos, severidad, detalle de cada petición, configuración de capas) y chat de demostración con un interruptor Firewall ON/OFF.

5. **Datos y despliegue**: eventos y configuración en una base de datos no relacional (DynamoDB), documentos en S3, vectores en Qdrant y despliegue en AWS con Terraform y CI/CD con GitHub Actions.

2.4 Tecnologías y coste

Python, FastAPI, Hugging Face, Presidio, sentence-transformers y Ollama para la IA; DynamoDB, S3 y Qdrant para los datos; AWS (Lambda, Cognito, CloudFront, CloudWatch) para la nube; Docker, Terraform y GitHub Actions para el despliegue; React para la interfaz web. El proyecto debe costar 0 €: el desarrollo se hará en local con modelos abiertos, y AWS se usará con el plan gratuito y sus créditos, creando la cuenta en la fase de despliegue y con un `docker-compose` como alternativa local para la defensa.

2.5 Alcance

Incluido: gateway con las cuatro capas de detección y filtro de salida; asistente RAG con permisos por rol; registro de eventos; dashboard y chat con interruptor del firewall; autenticación con roles; red teaming con métricas; despliegue en AWS; memoria, repositorio público y vídeo de demostración.

Opcional, según el tiempo: análisis de documentos al ingerirlos (indirect prompt injection), modo CTF con niveles, agente con herramientas y políticas, alertas por correo y comparativa con Amazon Bedrock Guardrails.

Fuera del alcance: uso en producción con datos reales, entrenamiento de modelos desde cero, garantía de detección del 100 % de los ataques (se mide y se reporta la eficacia) y pruebas contra sistemas de terceros. Aplicación móvil, SDK instalable y evaluación con LLM-as-judge quedan como trabajo futuro.

3. Requisitos funcionales

Prioridad: **I** = imprescindible, **D** = deseable.

ID	Requisito	Prioridad
RF-01	El sistema expondrá una API compatible con la de OpenAI que actúe como proxy hacia el modelo de lenguaje.	I
RF-02	El sistema analizará cada petición con un pipeline de detección por capas antes de enviarla al modelo.	I

ID	Requisito	Prioridad
RF-03	El sistema detectará ataques conocidos mediante reglas configurables.	I
RF-04	El sistema detectará y podrá enmascarar datos sensibles (DNI, IBAN, correo, teléfono, claves de API) en peticiones y respuestas.	I
RF-05	El sistema clasificará las peticiones con un modelo de IA para detectar prompt injection y jailbreak.	I
RF-06	El sistema asignará a cada petición una puntuación de riesgo y una decisión: permitir, enmascarar o bloquear.	I
RF-07	El sistema analizará las respuestas del modelo y detectará fugas de las instrucciones internas y de datos sensibles.	I
RF-08	El sistema funcionará con modelos locales (Ollama) y, si el coste lo permite, con Amazon Bedrock.	I
RF-09	El sistema permitirá cargar e indexar documentos de una empresa ficticia para el asistente con RAG.	I
RF-10	El asistente responderá usando solo los documentos recuperados e indicará la fuente.	I
RF-11	Cada documento tendrá permisos por rol y el asistente solo usará los que el usuario pueda ver.	I
RF-12	Los usuarios se autenticarán con login y roles (administrador, analista, empleado).	I
RF-13	El sistema registrará cada petición y su resultado en una base de datos no relacional, con los datos sensibles ya enmascarados.	I
RF-14	El dashboard mostrará peticiones, ataques bloqueados por tipo y severidad, y el detalle de cada petición (capa que actuó y decisión).	I
RF-15	El administrador podrá activar o desactivar capas y ajustar umbrales desde la web.	I
RF-16	El chat de demostración incluirá un interruptor Firewall ON/OFF.	I

ID	Requisito	Prioridad
RF-17	El módulo de red teaming ejecutará una batería de ataques clasificados según OWASP Top 10 for LLM contra el asistente, con y sin firewall.	I
RF-18	El módulo calculará la tasa de detección, los falsos positivos y la latencia añadida, y generará un informe de resultados.	I
RF-19	El sistema analizará los documentos al ingerirlos para detectar instrucciones ocultas (indirect prompt injection).	D
RF-20	La web incluirá un modo CTF con niveles de dificultad creciente.	D
RF-21	El sistema incluirá un agente con herramientas simuladas y políticas sobre qué puede ejecutar, con aprobación humana en acciones críticas.	D
RF-22	El sistema enviará alertas por correo ante ataques de severidad alta.	D

4. Requisitos no funcionales

ID	Categoría	Requisito
RNF-01	Coste	El proyecto debe costar 0 €: desarrollo en local con modelos abiertos y AWS solo con el plan gratuito y sus créditos, con alertas de crédito activadas.
RNF-02	Seguridad	Comunicaciones por HTTPS, datos cifrados en reposo, secretos fuera del repositorio, permisos de AWS con mínimo privilegio y autenticación en todos los endpoints.
RNF-03	Privacidad	Solo se usarán datos ficticios y el diseño seguirá los principios del RGPD (minimización, enmascarado y control de acceso).
RNF-04	Ética y legalidad	Los ataques de prueba se dirigirán únicamente contra el propio sistema del proyecto.

ID	Categoría	Requisito
RNF-05	Rendimiento	Las capas de reglas y datos sensibles añadirán menos de 300 ms por petición (objetivo a validar) y la latencia total del pipeline se medirá y documentará.
RNF-06	Eficacia	La detección se evaluará con al menos 100 ataques y 100 peticiones legítimas, reportando detección y falsos positivos.
RNF-07	Fiabilidad	Si una capa avanzada falla, el sistema seguirá funcionando con las restantes y nunca devolverá información sensible en un error.
RNF-08	Mantenibilidad	Cada capa de detección será un módulo independiente y ampliable, con pruebas automatizadas y análisis de seguridad del código en CI.
RNF-09	Portabilidad	El sistema se empaquetará con Docker y Terraform para desplegarse igual en local y en AWS, con un `docker-compose` como alternativa para la defensa.
RNF-10	Usabilidad	La interfaz será responsive y una aplicación compatible con OpenAI se integrará cambiando solo la URL base.
RNF-11	Documentación	El repositorio tendrá README, diagrama de arquitectura, guía de instalación y documentación de la API.
RNF-12	Trabajo en equipo	Se usará Git con ramas y revisión de cambios entre los miembros, y un tablero de tareas gestionado por la Team Lead.